

Competitive Programming

Section 36 Editorial: Fenwick Trees

Competitive Programming Course

2026-08-28

Table of contents

1	Editorial overview	4
2	A. Point Add Range Sum	5
2.1	Restatement	5
2.2	Algorithm	5
2.3	Correctness proof	5
2.4	Complexity	5
2.5	Full C++ solution	5
2.6	Full Python solution	6
3	B. Inversion Count	8
3.1	Restatement	8
3.2	Algorithm	8
3.3	Correctness proof	8
3.4	Complexity	8
3.5	Full C++ solution	8
3.6	Full Python solution	9
4	C. Range Add Point Query	11
4.1	Restatement	11
4.2	Algorithm	11
4.3	Correctness proof	11
4.4	Complexity	11
4.5	Full C++ solution	11
4.6	Full Python solution	12
5	D. Offline Distinct Range Queries	14
5.1	Restatement	14
5.2	Constraints and traps	14
5.3	Observations	14
5.4	Sample walk-through	15
5.5	Algorithm	15
5.6	Correctness proof	15
5.7	Complexity	16
5.8	C++ implementation notes	16
5.9	Python implementation notes	16
5.10	Full C++ solution	16
5.11	Full Python solution	18
5.12	Common wrong answers	19
5.13	Tests that matter	19
6	Testing discussion	20
7	Problem-specific upsolve notes	21
7.1	A. Point Add Range Sum	21

7.2	B. Inversion Count	21
7.3	C. Range Add Point Query	22
7.4	D. Offline Distinct Range Queries	22
8	Stress-testing workflow	23

1 Editorial overview

Problem	Main idea
A. Point Add Range Sum	See full algorithm and code below.
B. Inversion Count	See full algorithm and code below.
C. Range Add Point Query	See full algorithm and code below.
D. Offline Distinct Range Queries	Sweep by right endpoint and keep only latest occurrences active.

2 A. Point Add Range Sum

2.1 Restatement

Maintain an array. Operation 1 $i\ x$ adds x to $a[i]$. Operation 2 $l\ r$ asks for the sum of $a[l..r]$.

2.2 Algorithm

Identify the operation that must be fast, store exactly the aggregate information needed for that operation, and update the affected logarithmic frontier. The lesson for this section explains the invariant in detail; the implementation below is a direct transcription of that invariant.

For this problem, every query is processed in the order given except offline queries explicitly marked as offline. The answer is emitted immediately when the operation asks for one.

2.3 Correctness proof

The maintained structure stores the exact summary promised by the lesson: prefix blocks for Fenwick trees, segment aggregates for segment trees, power-of-two jumps for static structures, or the active frontier for heap/Mo-style processing. Each update changes precisely the summaries whose represented set contains the changed element or interval. Each query decomposes the requested object into summaries that are disjoint, idempotently overlapping, or maintained as the current active window. The merge formula for those summaries is exactly the mathematical definition of the requested answer, so every printed value is correct. $\square$

2.4 Complexity

The optimized operations are logarithmic unless the lesson says the structure is static or offline. Memory is linear or $O(n \log n)$ for sparse-table/binary-lifting preprocessing.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct Fenwick {
    int n;
    vector<long long> bit;
```

```

Fenwick(int n) : n(n), bit(n + 1, 0) {}
void add(int i, long long delta) {
    for (; i <= n; i += i & -i) bit[i] += delta;
}
long long sum(int i) const {
    long long res = 0;
    for (; i > 0; i -= i & -i) res += bit[i];
    return res;
}
long long range_sum(int l, int r) const {
    return sum(r) - sum(l - 1);
}
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    if (!(cin >> n >> q)) return 0;
    Fenwick bit(n);
    for (int i = 1; i <= n; i++) {
        long long x;
        cin >> x;
        bit.add(i, x);
    }
    while (q--) {
        int typ;
        cin >> typ;
        if (typ == 1) {
            int i;
            long long x;
            cin >> i >> x;
            bit.add(i, x);
        } else {
            int l, r;
            cin >> l >> r;
            cout << bit.range_sum(l, r) << '\n';
        }
    }
    return 0;
}

```

2.6 Full Python solution

```

import sys

class Fenwick:
    def __init__(self, n):

```

```

        self.n = n
        self.bit = [0] * (n + 1)
    def add(self, i, delta):
        while i <= self.n:
            self.bit[i] += delta
            i += i & -i
    def sum(self, i):
        res = 0
        while i > 0:
            res += self.bit[i]
            i -= i & -i
        return res
    def range_sum(self, l, r):
        return self.sum(r) - self.sum(l - 1)

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, q = data[0], data[1]
    a = data[2:2 + n]
    bit = Fenwick(n)
    for i, x in enumerate(a, 1):
        bit.add(i, x)
    idx = 2 + n
    out = []
    for _ in range(q):
        typ = data[idx]
        idx += 1
        if typ == 1:
            i, x = data[idx], data[idx + 1]
            idx += 2
            bit.add(i, x)
        else:
            l, r = data[idx], data[idx + 1]
            idx += 2
            out.append(str(bit.range_sum(l, r)))
    print("\n".join(out))

if __name__ == "__main__":
    main()

```

3 B. Inversion Count

3.1 Restatement

Given an array, count pairs $i < j$ with $a[i] > a[j]$. Values may repeat and may be large.

3.2 Algorithm

Identify the operation that must be fast, store exactly the aggregate information needed for that operation, and update the affected logarithmic frontier. The lesson for this section explains the invariant in detail; the implementation below is a direct transcription of that invariant.

For this problem, every query is processed in the order given except offline queries explicitly marked as offline. The answer is emitted immediately when the operation asks for one.

3.3 Correctness proof

The maintained structure stores the exact summary promised by the lesson: prefix blocks for Fenwick trees, segment aggregates for segment trees, power-of-two jumps for static structures, or the active frontier for heap/Mo-style processing. Each update changes precisely the summaries whose represented set contains the changed element or interval. Each query decomposes the requested object into summaries that are disjoint, idempotently overlapping, or maintained as the current active window. The merge formula for those summaries is exactly the mathematical definition of the requested answer, so every printed value is correct. $\square$

3.4 Complexity

The optimized operations are logarithmic unless the lesson says the structure is static or offline. Memory is linear or $O(n \log n)$ for sparse-table/binary-lifting preprocessing.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct Fenwick {
    int n;
    vector<int> bit;
    Fenwick(int n) : n(n), bit(n + 1, 0) {}
};
```

```

void add(int i, int delta) {
    for (; i <= n; i += i & -i) bit[i] += delta;
}
int sum(int i) const {
    int res = 0;
    for (; i > 0; i -= i & -i) res += bit[i];
    return res;
}
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    vector<long long> a(n), vals;
    for (long long &x : a) {
        cin >> x;
        vals.push_back(x);
    }
    sort(vals.begin(), vals.end());
    vals.erase(unique(vals.begin(), vals.end()), vals.end());
    Fenwick bit((int)vals.size());
    long long inv = 0;
    for (int i = 0; i < n; i++) {
        auto position = lower_bound(vals.begin(), vals.end(), a[i]);
        int rank = int(position - vals.begin()) + 1;
        inv += i - bit.sum(rank);
        bit.add(rank, 1);
    }
    cout << inv << '\n';
    return 0;
}

```

3.6 Full Python solution

```

import sys

class Fenwick:
    def __init__(self, n):
        self.n = n
        self.bit = [0] * (n + 1)
    def add(self, i, delta):
        while i <= self.n:
            self.bit[i] += delta
            i += i & -i
    def sum(self, i):
        res = 0

```

```

        while i > 0:
            res += self.bit[i]
            i -= i & -i
        return res

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n = data[0]
    a = data[1:1 + n]
    values = {x: i + 1 for i, x in enumerate(sorted(set(a)))}
    bit = Fenwick(len(values))
    inv = 0
    seen = 0
    for x in a:
        rank = values[x]
        inv += seen - bit.sum(rank)
        bit.add(rank, 1)
        seen += 1
    print(inv)

if __name__ == "__main__":
    main()

```

4 C. Range Add Point Query

4.1 Restatement

Maintain an array. Operation 1 $l\ r\ x$ adds x to every element in $[l, r]$. Operation 2 i asks for the current value of $a[i]$.

4.2 Algorithm

Identify the operation that must be fast, store exactly the aggregate information needed for that operation, and update the affected logarithmic frontier. The lesson for this section explains the invariant in detail; the implementation below is a direct transcription of that invariant.

For this problem, every query is processed in the order given except offline queries explicitly marked as offline. The answer is emitted immediately when the operation asks for one.

4.3 Correctness proof

The maintained structure stores the exact summary promised by the lesson: prefix blocks for Fenwick trees, segment aggregates for segment trees, power-of-two jumps for static structures, or the active frontier for heap/Mo-style processing. Each update changes precisely the summaries whose represented set contains the changed element or interval. Each query decomposes the requested object into summaries that are disjoint, idempotently overlapping, or maintained as the current active window. The merge formula for those summaries is exactly the mathematical definition of the requested answer, so every printed value is correct. $\square$

4.4 Complexity

The optimized operations are logarithmic unless the lesson says the structure is static or offline. Memory is linear or $O(n \log n)$ for sparse-table/binary-lifting preprocessing.

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct Fenwick {
    int n;
    vector<long long> bit;
```

```

Fenwick(int n) : n(n), bit(n + 2, 0) {}
void add(int i, long long delta) {
    for (; i <= n; i += i & -i) bit[i] += delta;
}
long long sum(int i) const {
    long long res = 0;
    for (; i > 0; i -= i & -i) res += bit[i];
    return res;
}
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    if (!(cin >> n >> q)) return 0;
    Fenwick bit(n + 1);
    long long prev = 0;
    for (int i = 1; i <= n; i++) {
        long long x;
        cin >> x;
        bit.add(i, x - prev);
        prev = x;
    }
    while (q--) {
        int typ;
        cin >> typ;
        if (typ == 1) {
            int l, r;
            long long x;
            cin >> l >> r >> x;
            bit.add(l, x);
            bit.add(r + 1, -x);
        } else {
            int i;
            cin >> i;
            cout << bit.sum(i) << '\n';
        }
    }
    return 0;
}

```

4.6 Full Python solution

```

import sys

class Fenwick:
    def __init__(self, n):

```

```

        self.n = n
        self.bit = [0] * (n + 2)
    def add(self, i, delta):
        while i <= self.n:
            self.bit[i] += delta
            i += i & -i
    def sum(self, i):
        res = 0
        while i > 0:
            res += self.bit[i]
            i -= i & -i
        return res

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, q = data[0], data[1]
    a = data[2:2 + n]
    bit = Fenwick(n + 1)
    prev = 0
    for i, x in enumerate(a, 1):
        bit.add(i, x - prev)
        prev = x
    idx = 2 + n
    out = []
    for _ in range(q):
        typ = data[idx]
        idx += 1
        if typ == 1:
            l, r, x = data[idx], data[idx + 1], data[idx + 2]
            idx += 3
            bit.add(l, x)
            bit.add(r + 1, -x)
        else:
            i = data[idx]
            idx += 1
            out.append(str(bit.sum(i)))
    print("\n".join(out))

if __name__ == "__main__":
    main()

```

5 D. Offline Distinct Range Queries

5.1 Restatement

For every static interval $[l, r]$, count how many different values occur in that interval. The queries do not modify the array, so they may be processed offline in a more useful order and then returned to their original order.

5.2 Constraints and traps

Building a set separately for every query can inspect $O(nq)$ elements. A normal Fenwick tree can count marked positions, but marking every occurrence would count occurrences rather than distinct values. We need a marking rule that leaves exactly one active position for each relevant value.

5.3 Observations

Sort the queries by right endpoint and sweep the array from left to right. After processing position r , maintain this invariant:

```
For every value seen in  $a[1..r]$ :  
    exactly its latest occurrence has weight 1.  
Every other processed position has weight 0.
```

When the sweep reaches position r with value x :

```
if  $x$  previously appeared at  $old$ :  
    add( $old$ , -1)  
add( $r$ , +1)  
last[ $x$ ] =  $r$ 
```

Now consider a query $[l, r]$. Each value occurring in the range has a latest occurrence at some position from l through r , so it contributes exactly one active marker to that range. A value absent from the range has no active marker there. Therefore the number of distinct values is simply:

```
prefix( $r$ ) - prefix( $l-1$ )
```

5.4 Sample walk-through

For the prefix $[1,5]$ of $a = [1,2,1,3,2,4,1]$, the latest occurrences are:

```
value 1 -> position 3
value 2 -> position 5
value 3 -> position 4

position      1  2  3  4  5
active marker 0  0  1  1  1
```

The query $[2,5]$ sums those markers and gets 3. When the sweep reaches position 7, value 1 moves from position 3 to position 7: subtract one at 3, then add one at 7.

5.5 Algorithm

```
attach each query's original index
sort queries by increasing right endpoint
processed_right = 0

for each query (l, r) in that order:
    while processed_right < r:
        advance processed_right
        x = a[processed_right]
        if x has an old latest position:
            add(old_position[x], -1)
        add(processed_right, +1)
        old_position[x] = processed_right
    answer[original_index] = prefix(r) - prefix(l-1)

print answers in original order
```

5.6 Correctness proof

The invariant is true after the first position: its value has one marker at its latest occurrence. When processing a new position with value x , removing the old marker for x and adding the new one preserves exactly one marker at the latest occurrence of x ; markers for all other values are unchanged. Thus the invariant holds after every sweep step.

For a query $[l,r]$, take any value occurring in the interval. Its latest occurrence in the processed prefix $[1,r]$ cannot be before l , because it has an occurrence inside $[l,r]$; hence its unique marker is counted. Conversely, any marker counted inside $[l,r]$ belongs to a value occurring there. The range sum therefore counts each distinct value exactly once. Restoring answers by their original indices produces the required output order. $\square$

5.7 Complexity

Sorting the queries costs $O(q \log q)$. The sweep performs at most two Fenwick updates per array position and two prefix queries per range:

```
Time:     $O(q \log q + (n+q) \log n)$   
Memory:  $O(n+q)$ 
```

5.8 C++ implementation notes

Store the original query index before sorting. `unordered_map<long long,int>` can track the latest position without coordinate compression; reserve space in advance to avoid repeated rehashing on large inputs.

5.9 Python implementation notes

Tuples `(right,left,index)` sort into the required sweep order directly. A dictionary maps arbitrary array values to their latest 1-based positions.

5.10 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct Fenwick {
    int n;
    vector<int> bit;

    explicit Fenwick(int size) : n(size), bit(size + 1, 0) {}

    void add(int index, int delta) {
        for (; index <= n; index += index & -index) {
            bit[index] += delta;
        }
    }

    int sum(int index) const {
        int result = 0;
        for (; index > 0; index -= index & -index) {
            result += bit[index];
        }
        return result;
    }

    int range_sum(int left, int right) const {
        return sum(right) - sum(left - 1);
    }
}
```

```

    }
};

struct Query {
    int left;
    int right;
    int index;
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) return 0;
    vector<long long> values(n + 1);
    for (int i = 1; i <= n; ++i) cin >> values[i];

    vector<Query> queries(q);
    for (int i = 0; i < q; ++i) {
        cin >> queries[i].left >> queries[i].right;
        queries[i].index = i;
    }
    sort(
        queries.begin(),
        queries.end(),
        [](const Query& a, const Query& b) {
            return a.right < b.right;
        }
    );

    Fenwick active(n);
    unordered_map<long long, int> last_position;
    last_position.reserve(2 * n);
    vector<int> answer(q);
    int processed_right = 0;

    for (const Query& query : queries) {
        while (processed_right < query.right) {
            ++processed_right;
            long long value = values[processed_right];
            auto previous = last_position.find(value);
            if (previous != last_position.end()) {
                active.add(previous->second, -1);
            }
            active.add(processed_right, 1);
            last_position[value] = processed_right;
        }
        answer[query.index] = active.range_sum(
            query.left,
            query.right

```

```

    );
}

for (int value : answer) cout << value << '\n';
return 0;
}

```

5.11 Full Python solution

```

import sys

class Fenwick:
    def __init__(self, size: int) -> None:
        self.size = size
        self.bit = [0] * (size + 1)

    def add(self, index: int, delta: int) -> None:
        while index <= self.size:
            self.bit[index] += delta
            index += index & -index

    def sum(self, index: int) -> int:
        result = 0
        while index > 0:
            result += self.bit[index]
            index -= index & -index
        return result

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return

    n, q = data[0], data[1]
    values = data[2 : 2 + n]
    offset = 2 + n
    queries = []
    for query_index in range(q):
        left, right = data[offset], data[offset + 1]
        offset += 2
        queries.append((right, left, query_index))
    queries.sort()

    bit = Fenwick(n)
    last_position: dict[int, int] = {}
    answers = [0] * q

```

```

processed_right = 0

for right, left, query_index in queries:
    while processed_right < right:
        value = values[processed_right]
        position = processed_right + 1
        previous = last_position.get(value)
        if previous is not None:
            bit.add(previous, -1)
        bit.add(position, 1)
        last_position[value] = position
        processed_right += 1
    answers[query_index] = bit.sum(right) - bit.sum(left - 1)

sys.stdout.write("\n".join(map(str, answers)) + "\n")

if __name__ == "__main__":
    main()

```

5.12 Common wrong answers

Keeping all occurrences active:
 this counts elements instead of values.
 Adding the new occurrence without removing the previous one.
 Answering a query before the sweep has reached its right endpoint.
 Printing answers in sorted-query order instead of input order.
 Using a set per query:
 this is quadratic on large overlapping ranges.

5.13 Tests that matter

All values equal: every nonempty range answers 1.
 All values different: every answer is $r-l+1$.
 Repeated values whose latest occurrence repeatedly moves.
 Queries sharing a right endpoint.
 Queries supplied in reverse endpoint order.
 $n=q=200,000$ with long ranges, to reject rebuilding a set per query.

6 Testing discussion

The fixed samples cover visible behavior. The random tests generate many small instances and use literal arrays or sets as independent oracles, then the same expected files are used against both the C++ and Python submissions. For D, a maximum-size periodic array has a closed-form answer for every generated range, so the complexity test does not depend on a slow oracle.

7 Problem-specific upsolve notes

7.1 A. Point Add Range Sum

Constraint implication and trap. With a long sequence of updates and queries, scanning $[1, r]$ per query is quadratic in the worst case. The update is an addition, not an assignment: call `add(i, x)`, never overwrite the old Fenwick contribution.

Walk-through. From $[1, 2, 3, 4]$, query $[2, 4]$ is `prefix(4)-prefix(1)=9`. After `add(3, 10)`, the same query is 19. The tree's prefix invariant proves the subtraction removes exactly positions before 1.

```
build: for i=1..n: add(i, a[i])
type 1 i x: add(i, x)
type 2 l r: print prefix(r)-prefix(l-1)
```

Every `bit[j]` stores its lowbit block sum. Prefix blocks partition $[1, r]$ and $[1, l-1]$, so their difference is exactly $[l, r]$. Build is $O(n \log n)$ with these calls and each operation is $O(\log n)$; memory is $O(n)$. In both C++ and Python, use 64-bit-capable sums. Test $l=1$, $l=r$, negative delta, and a query after several additions to one position.

7.2 B. Inversion Count

Observation. Values can be large or negative, so compress them to ordered ranks. At index i , $i - \text{prefix}(\text{rank})$ is the number of earlier values that are strictly greater; using `prefix(rank-1)` would incorrectly count equals.

For $[3, 1, 2, 5, 4]$, the contributions are $0, 1, 1, 0, 1$, total 3. The Fenwick invariant counts exactly the processed values of each rank, and compression preserves $<$, proving the accumulated contributions count every inversion once when its right endpoint arrives. Complexity is $O(n \log n)$ time and $O(n)$ memory. C++ needs `long long inv`; Python needs no special numeric type. Test sorted, reverse-sorted, all-equal, and values outside 32-bit range.

```
compress values to ranks
for i from 0 to n-1:
    answer += i - prefix(rank[a[i]])
    add(rank[a[i]], 1)
```

7.3 C. Range Add Point Query

Observation. Store `diff`, not the logical values. A range add changes only `diff[l]` and `diff[r+1]`; a point value is the prefix sum of differences. For `[10,20,30,40]`, add 5 on `[2,3]`: prefix at 3 changes from 30 to 35, but prefix at 4 returns to 40 because of `diff[4]=-5`.

The difference identity proves that every position in `[1,r]` sees `+x` once and every other position sees net zero. Each update and point query is $O(\log n)$, with $O(n)$ memory. Allocate an extra usable index for `r+1`. In C++/Python, initialize by inserting `a[i]-a[i-1]`, not `a[i]`. Test a full-range add, `r=n`, and a negative update.

7.4 D. Offline Distinct Range Queries

Observation. Offline order turns a two-dimensional query condition into a one-dimensional sweep. Once all positions through `r` are processed, keeping only the latest occurrence of each value active makes a Fenwick range sum count values rather than occurrences.

The crucial update is a move, not an insertion: deactivate the old latest position before activating the new one. A query `[l,r]` then counts exactly the latest occurrences lying at or after `l`. Preserve each query's input index so sorting by `r` does not reorder the output. Complexity is $O(q \log q + (n+q) \log n)$ time and $O(n+q)$ memory. Test repeated values, queries with the same right endpoint, and input queries ordered by decreasing right endpoint.

8 Stress-testing workflow

For A and C, keep a slow list and mutate it literally; for B, use a double loop on arrays of length at most eight. For D, compare every range against the size of a literal set, then add a maximum periodic case whose answer is `min(range_length, period)`. Compare after every random operation, then run `CP_TARGET=solution python3 sections/36_fenwick_trees/check.py`. Carry forward: Fenwick trees are prefix structures, and coordinate compression, a difference transform, or an offline sweep changes what that prefix means.